

Practice Set: Static Members in C++

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define a **Static Local Variable** in C++. Compare an automatic (normal) local variable with a static local variable in terms of memory segment allocation, default initialization, and lifetime across function invocations.

Question 2: Explain what a **Static Member Variable (Class Variable)** is in C++. How does its allocation and copy count differ from an Instance Member Variable when 10 objects of a class are instantiated?

Question 3: Detail the memory architecture of a class containing both an instance variable and a static member variable (e.g., class Account { int balance; static float roi; };). Where is each variable stored in RAM, and how much memory does each instantiated object occupy?

Question 4: State the **Cardinal Rule** regarding the external definition of static member variables in C++. What is the syntax for this definition, and why is declaring it inside the class body alone insufficient?

Question 5: If a static member variable is declared under the public section, describe the two syntactical ways to access or update it. Which approach is recommended as best practice, and why?

Question 6: What limitation arises when trying to access or modify a private static member variable before any objects of that class are instantiated? How do **Static Member Functions** solve this problem?

Question 7: Detail the rules and constraints governing **Static Member Functions** in C++. Why can't a static member function access non-static instance variables or call non-static member functions?

Question 8: Compare **Instance Members (Non-Static)** versus **Static Members (Class-Level)** across six criteria: Memory Allocation, Number of Copies, Object Dependency, Invocation Method, this Pointer Availability, and Access Capabilities.

Question 9: Identify common architectural use cases for static member variables and static functions in professional C++ software design.

Question 10: Write a complete C++ program defining an Account class that encapsulates a private instance variable balance and a private static member variable roi. Implement appropriate instance member functions and static member functions to set and display both values, demonstrating that static member functions can be invoked before any object is created.

Section 2: Interview & Viva Questions (11-20)

Question 11 (Viva): In which memory segment are static variables (both local and class-level) stored in RAM?

Question 12 (Viva): What default value is assigned to an uninitialized static variable in C++?

Question 13 (Viva): Does a static member function receive an implicit this pointer when invoked?

Question 14 (Viva): What linker/compiler error occurs if a developer declares a static member variable inside a class but forgets to define it outside at file scope?

Question 15 (Viva): If sizeof(int) is 4 bytes and sizeof(float) is 4 bytes, what is the result of sizeof(a1) for an object of class Account { int balance; static float roi; }?

Question 16 (Viva): Can a static member function be declared as a virtual function in C++?

Question 17 (Viva): Can a non-static (instance) member function read or modify a static member variable?

Question 18 (Viva): What happens to the value seen by other objects of a class if one object modifies a shared public static member variable (e.g., a1.roi = 5.0f;)?

Question 19 (Viva): Why is calling a static member function via the class name (e.g., Account::getRoi()) preferred over calling it through an object instance (e.g., a1.getRoi())?

Question 20 (Viva): Can a static local variable inside a function be accessed directly by name outside that function's scope?

Answers & Explanations

Answer: 01

Definition: A Static Local Variable is a local variable declared inside a function using the static keyword that retains its value across multiple function calls.

Comparison Matrix:

Memory Segment: Automatic local variables are allocated dynamically on the Call Stack; static local variables are allocated in the Data/BSS Segment.

Default Initialization: Automatic variables hold indeterminate garbage bits; static local variables are automatically zero-initialized (0 or nullptr) by default.

Lifetime: Automatic variables are created on function entry and destroyed at block exit; static local variables are created once at program startup and persist until application termination.

Answer: 02

Definition: A Static Member Variable (Class Variable) is a class-level variable declared using static that belongs to the class itself rather than any individual object.

Allocation & Copies:

For instance variables, each of the 10 objects receives its own independent copy in RAM.

For static member variables, exactly **one single shared copy** is created in the Data Segment for the entire class, shared universally by all 10 objects.

Answer: 03

Memory Placement:

balance (Instance Variable): Allocated on the Stack (or Heap) inside each individual object's memory block.

roi (Static Member Variable): Allocated once in the application's Data Segment at a fixed memory address independent of object instances.

Object Size: Each instantiated Account object occupies **4 Bytes** (allocating storage solely for balance). The static roi is not part of any object's memory layout.

Answer: 04

Cardinal Rule: Declaring a static member variable inside a class only declares its existence; it does not allocate memory storage. It must be explicitly defined outside the class at file scope using the Scope Resolution Operator (::).

Syntax:

```
data_type ClassName::variable_name = initial_value;
```

Reason: Without this external definition, the compiler and linker do not reserve physical storage in the Data Segment, resulting in an undefined symbol link error.

Answer: 05

Two Syntactical Approaches:

Through Class Name: Account::roi = 6.5f;

Through Object Name: a1.roi = 6.5f;

Recommended Best Practice: Accessing via the Class Name

(ClassName::variable) is preferred because it explicitly communicates that the variable is a shared class-level property and works even when zero objects exist. Accessing via an object creates the false impression that only that object's state is modified.

Answer: 06

The Limitation: Private static variables cannot be accessed directly outside the class interface. If no objects are created, standard instance member functions cannot be invoked to read or modify them.

The Solution: Static Member Functions belong to the class and can be invoked directly through the class name (ClassName::function()) without requiring any object instance.

Answer: 07

Rules & Constraints

Can be called without any object using the class name and ::.

Can only access static variables or other static functions in the class directly.

Cannot access non-static instance variables or call non-static member functions.

Operates without an implicit this pointer.

Reason: Because static functions can run without an object instance, they have no this pointer pointing to a caller object in RAM, making it impossible to resolve which object's instance variable should be accessed.

Answer: 08

Feature	Instance Member (Non-Static)	Static Member (Class-Level)
Memory Allocation	Allocated per object on Stack/Heap at instantiation.	Allocated once in the Data Segment at program start.
Number of Copies	One unique copy per instantiated object.	Exactly one shared copy for the entire class.
Object Dependency	Dependent on object existence.	Independent; exists even with zero objects.
Invocation Method	Called via object instance (obj.func()).	Called via Class Name (Class::func()).
this Pointer	Has implicit this pointer to caller object.	No this pointer.
Access Capabilities	Can access both instance and static members.	Can access only static members directly.

Answer: 09

Shared Global / Class Configurations: Storing common parameters like bank interest rates or app configuration flags.

Object Instance Counters: Tracking the total count of active sessions, connections, or created objects.

Design Patterns: Implementing Singleton design patterns or factory utility classes.

Answer: 10

```
#include <iostream>
using namespace std;
class Account {private:
    int balance;    // Instance Variable
    static float roi; // Static Member Variable
public:
    void setBalance(int b)
    {
        balance = b;
    }

    void displayAccount()
    {
        cout << "Account Balance: $" << balance << " | ROI: " << roi << "%" << endl;
    }

    // Static Member Functions (Callable without objects)
    static void setRoi(float r)
    {
        roi = r; // Can only modify static members
    }

    static float getRoi()
    {
        return roi;
    }
};
// Mandatory external definition allocating RAM in Data Segment float Account::roi =
4.0f;
int main()
{
    // 1. Calling Static Function BEFORE instantiating objects
    cout << "Initial Bank ROI: " << Account::getRoi() << "%" << endl;
    Account::setRoi(5.5f);

    // 2. Instantiating Account Objects
    Account a1, a2;
    a1.setBalance(10000);
    a2.setBalance(25000);

    cout << "\nAccount Details:" << endl;
    a1.displayAccount();
    a2.displayAccount();

    // 3. Modifying shared static rate updates both instances
    Account::setRoi(6.0f);
    cout << "\nAfter Bank-wide ROI Update to 6.0%:" << endl;
```

```
a1.displayAccount();
a2.displayAccount();

return 0;
}
```

Answer: 11

In the **Data / BSS Segment** of memory (Global/Static Storage).

Answer: 12

It is automatically zero-initialized (0 for numeric types, nullptr for pointers).

Answer: 13

No, static member functions do not receive a this pointer because they are not bound to an individual object instance.

Answer: 14

An **"undefined reference"** linker error, because memory was never reserved in the Data Segment.

Answer: 15

4 bytes. The static member roi is stored separately in the Data Segment and does not contribute to the object's stack size.

Answer: 16

No, static member functions cannot be virtual because virtual dispatch relies on runtime object instances and virtual tables (vtables), while static functions lack an object context.

Answer: 17

Yes, non-static instance member functions have full access to both instance variables and static member variables.

Answer: 18

Because the variable is shared across the entire class, modifying it through one object instantly updates the value seen by all other instances.

Answer: 19

Because calling via `ClassName::func()` makes it clear that the operation affects class-level state rather than instance state, and it functions even when zero objects exist.

Answer: 20

No; while its lifetime extends until program termination, its scope (visibility) remains strictly local to the defining function block.